

QUESTIONS: 10 / 10 completed

Stack using Array

STACK USING ARRAY

"A web browser maintains a history of recently visited pages. Write a C program to implement a stack using an array and perform Push, Pop, Peek, and Display operations while handling overflow and underflow conditions. Test Case 1

Input

Stack Size = 5
Push: 10, 20, 30
Display

Output

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 #define MAX 5
3
4 int main(){
5     int stack[MAX],top=-1,n,x;
6
7     printf("enter number of elements:");
8     scanf("%d",&n);
9
10    printf("Enter elements:");
11    for(int i=0;i<n;i++){
12        scanf("%d",&x);
13        stack[++top]=x;
14    }
15
16    printf("Stack:");
17    while(top!=-1)
18        printf("%d ",stack[top--]);
19
20    return 0;
21 }
```

OUTPUT

```
enter number of elements:Enter elements:Stack:50 40 30 20 10
```

INPUT

```
5
10 20 30 40 50
```

QUESTIONS 10 / 10 completed

Stack using Linked

STACK USING LINKED LIST

"A document editor maintains recently performed actions to support the Undo feature. Implement a stack using a linked list and perform push, pop, and display operations. Test Case 1

Input

Push: 100, 200, 300

Display

Output

300 → 200 → 100 → NULL"

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{
5     int data;
6     struct node*next;
7 };
8 int main(){
9     struct node*top=NULL,*new;
10    int n,x;
11
12    printf("enter number of elements:");
13    scanf("%d",&n);
14
15    for(int i=0;i<n;i++){
16        scanf("%d",&x);
17        new=malloc(sizeof(struct node));
18        new->data=x;
19        new->next=top;
20        top=new;
21    }
```

OUTPUT

```
enter number of elements:Stack:40 30 20 10
```

INPUT

```
4
10 20 30 40
```

QUESTIONS 10 / 10 completed

Stack -Postfix Expr

STACK -POSTFIX EXPRESSION

"Write a C program that uses a stack to evaluate a postfix expression and display the result.

Postfix expressions are commonly used in compilers because they eliminate the need for parentheses.

Input

23*5+

Output

Result = 11

..

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[20];
6     int top = -1;
7     int x, y, ans;
8     char ch;
9
10    while (1)
11    {
12        ch = getchar();
13
14        if (ch == '\n')
15            break;
16
17        if (ch >= '0' && ch <= '9')
18        {
19            x = ch - '0';
20            top = top + 1;
21            a[top] = x;
```

OUTPUT

INPUT

23+

QUESTIONS 10 / 10 completed

Stack -Infix to Post

STACK -INFIX TO POSTFIX

"Arithmetic expressions entered by programmers are generally in infix form. Before evaluation, many systems convert them into postfix notation. Write a C program using a stack to convert an infix expression into its equivalent postfix expression. Test Case 1

Input

A+B*C

Output

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2
3 int main(){
4     char e[20];
5     int s[20],top=-1,i,a,b;
6
7     scanf("%s",e);
8
9     for(i=0;e[i]!='\0';i++)
10    {
11        if(e[i]>='0')
12
13    }
14 }
```

OUTPUT

Security Error: Disallowed code pattern detected.

INPUT

A+B*C

QUESTIONS 10 / 10 completed

Stack- Balanced Parentheses

STACK-BALANCED PARENTHESES

"Compilers must verify whether all opening parentheses have corresponding closing parentheses. Write a C program using a stack to check whether an expression contains balanced parentheses. Input

(A+B)*(C-D)

Output

Balanced Expression
"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char s[50], stack[50];
6     int top=-1, i, ok=1;
7
8     scanf("%s",s);
9
10    for(i=0;s[i]!='\0';i++)
11    {
12        if(s[i]=='(')
13            stack[++top]='(';
14        else if(s[i]==')')
15        {
16            if(top== -1)
17            {
18                ok=0;
19                break;
20            }
21            top--;
```

OUTPUT

Security Error: Disallowed code pattern detected.

INPUT

Hello

QUESTIONS 10 / 10 completed

Stack - Reverse Str

STACK - REVERSE STRING

"A text-processing application requires reversing user-entered strings. Write a C program that uses a stack to reverse a given string.
Input

DATASTRUCTURE

Output

ERUTCURISATAD

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char a[100];
6     int i = 0;
7     int c;
8
9     while (i < 99)
10    {
11        c = getchar();
12
13        if (c == EOF || c == '\n')
14            break;
15
16        a[i] = c;
17        i = i + 1;
18    }
19
20    for (i = i - 1; i >= 0; i = i - 1)
21    {
```

OUTPUT

ERUTCURISATAD

INPUT

DATASTRUCTURE

QUESTIONS 10 / 10 completed

Stack- Decimal to

STACK- DECIMAL TO BINARY

"Computer systems internally represent numbers in binary format. Write a C program that uses a stack to convert a decimal number into its binary equivalent.

Input:

13

Output:

1101"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, a[20], top = -1;
6
7     scanf("%d", &n);
8
9     if (n == 0)
10    {
11        printf("0");
12    }
13    else
14    {
15        while (n > 0)
16        {
17            top = top + 1;
18            a[top] = n % 2;
19            n = n / 2;
20        }
21    }
```

OUTPUT

1101

INPUT

13

QUESTIONS 10 / 10 completed

Tower of Hanoi Pr. ▾

TOWER OF
HANOI PROBLEM

"The Tower of Hanoi is a classic problem demonstrating recursion and stack behavior. Write a C program to solve the Tower of Hanoi problem and display the sequence of moves required to transfer all disks.

Input

Number of Disks = 3

Output

Move Disk 1 from A to C
Move Disk 2 from A to B
Move Disk 1 from C to B
Move Disk 3 from A to C
Move Disk 1 from B to A
Move Disk 2 from B to C
Move Disk 1 from A to C

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 void hanoi(int n, char from, char to, char temp)
4 {
5     if (n == 1)
6     {
7         printf("Move Disk 1 from %c to %c\n", from, to);
8         return;
9     }
10
11     hanoi(n - 1, from, temp, to);
12     printf("Move Disk %d from %c to %c\n", n, from, to);
13     hanoi(n - 1, temp, to, from);
14 }
15
16 int main()
17 {
18     int n;
19
20     scanf("%d", &n);
21 }
```

OUTPUT

```
Move Disk 1 from A to C
Move Disk 2 from A to B
Move Disk 1 from C to B
Move Disk 3 from A to C
Move Disk 1 from B to A
Move Disk 2 from B to C
Move Disk 1 from A to C
```

INPUT

3

QUESTIONS 10 / 10 completed

Stack - Browser Hi! ▾

STACK -
BROWSER
HISTORY

"A web browser stores visited web pages in a stack to enable navigation to previously visited pages. Write a C program that simulates browser history using a stack and supports back navigation."

Input:

Visit:

Google

YouTube

Wikipedia

Back:

Output:

Current Page: "YouTube"

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char stack[3][20];
6     int i, j, c;
7
8     for(i = 0; i < 3; i++)
9     {
10         j = 0;
11
12         for(; j < 19; j++)
13         {
14             c = getchar();
15
16             if(c == '\n' || c == EOF)
17                 break;
18
19             stack[i][j] = c;
20         }
21     }
```

OUTPUT

YouTube

INPUT

Google
YouTube
Wikipedia

QUESTIONS 10 / 10 completed

Stack- Undo Oper: ▾

STACK- UNDO OPERATION

"A text editor provides an Undo feature that allows users to revert the most recent action. Write a C program using a stack to simulate an Undo operation for text editing commands. Input

Insert A
Insert B
Insert C
Undo

Output

Current Text: AB"

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char a[3] = {'A', 'B', 'C'};
6     int top = 2;
7
8     top = top - 1;
9
10    printf("Undo: Insert %c", a[top + 1]);
11
12    return 0;
13 }
```

OUTPUT

Undo: Insert C

INPUT

Insert A
Insert B
Insert C
Undo